

DSAD Nota 7

Analiza in Componente Principale

Imports

```
import pandas as pd
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

Procesare/Pre-procesare

```
date = pd.read_csv("date_pca.csv")
print(date)

date_numerice = date.drop(columns={"Tara"}) # pentru pca avem nevoie
                                              # doar de coloanele numerice
print(date_numerice)

#standardizam datele
scaler = StandardScaler()
date_standard = scaler.fit_transform(date_numerice)

#aplicam PCA
pca = PCA()
date_pca = pca.fit(date_standard)
componente_principale = pca.components_
```

Varianță componente

```
varianta = pca.explained_variance_

# Procent varianta
var_ratio = pca.explained_variance_ratio_

# Varianta Cumulata
var_cumulata = np.cumsum(var_ratio)
```

Calcul componente și/sau scoruri

```
scoruri_componente = pca.transform(date_standard)
print("Componente principale:\n", componente_principale)
```

```
print("Scoruri componente:\n", scoruri_componente)
```

optional - graficul scorurilor pe 2 axe

```
plt.figure(figsize=(8,6))
plt.scatter(scoruri_componente[:,0], scoruri_componente[:,1],
            c="blue", alpha=0.8)
plt.title("Plot componente/scoruri")
plt.xlabel("Componenta principala 1")
plt.ylabel("Componenta principala 2")
plt.grid()
plt.show()
```

Analiza Factoriala

Importuri

```
import pandas as pd
from factor_analyzer import FactorAnalyzer
from factor_analyzer.factor_analyzer import calculate_bartlett_sphericity
from factor_analyzer.factor_analyzer import calculate_kmo
```

Pre-procesare

```
date = pd.read_csv("factorial_analysis.csv")
print(date)
x = date.iloc[:,0:] # selectam toate datele (daca avem index,[:, 1:])
print(x)
```

Analiza factorabilității - Bartlett

```
chi_square, p_value = calculate_bartlett_sphericity(x)
print("Factorabilitatea Bartlett")
print(f"Valoarea chi-square: {chi_square}")
print(f"P-Value: {p_value}") # vrem p_value < 0.05
```

Analiza factorabilității - KMO

```
kmo_all, kmo_model = calculate_kmo(x)
print(f"Scorul KMO total: {kmo_model}") # vrem kmo_model > 0.6
```

Calcul varianță factori (cu/fără rotație) - returneaza varianta, % de varianta, varianta cumulata

```
# fara rotatie? rotation=None
fa_cu_rotatie = FactorAnalyzer(n_factors=5, rotation="varimax")
fa_cu_rotatie.fit(x)

varianta_cu_rotatie = fa_cu_rotatie.get_factor_variance()
print(f"Varianta factori cu rotatie: {varianta_cu_rotatie}")
```

Calcul corelații factoriale (cu/fără rotație)

```
coreletii_factoriale_cu = fa_cu_rotatie.loadings_
print(f"Corelatii factoriale cu rotatie: {coreletii_factoriale_cu}")
```

Analiza Discriminanta

Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
```

Pre-procesare

```
date = pd.read_csv("discriminant_analysis_data.csv")
x = date.drop(columns=["Target"])
y = date["Target"]

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.3,
                                                    random_state=42)

lda = LinearDiscriminantAnalysis()
lda.fit(x_train, y_train)
```

Calcul scoruri discriminante model liniar

```
scoruri = lda.transform(x_test)
```

```
print("Scoruri discriminante\n", scoruri)
```

Trasare plot instanțe în axe discriminante

```
plt.figure(figsize=(10,8))
# pentru 2 axe: scoruri[:, 0], scoruri[:, 1]
plt.scatter(scoruri[:, 0], [0]*len(scoruri), c=y_test,
            cmap="viridis", edgecolors='k', s=50)
plt.title("Instante in axe discriminante")
plt.xlabel("Axa discriminanta 1")
# 2 axe - punem ylabel pentru axa2
plt.yticks([]) # pentru 2 axe, nu avem nevoie de asta
plt.colorbar(label="Clasa")
plt.show()
```

Analiza de clusteri

Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram
```

Pre-procesare

```
date = pd.read_csv("date_clusterizare.csv")
```

Calcul ierarhie (matricea ierarhie - col 1-2: clusterii uniti, col 3: distanta, col 4: nr elemente noi)

```
z = linkage(date, method="ward")
print("Matrice ierarhie:\n", z)

#si o dendrograma asa frumos - optional
plt.figure(figsize=(10,7))
dendrogram(z)
plt.title("Dendrograma - Calcul Ierarhie")
plt.xlabel("Instante")
plt.ylabel("Distanta")
plt.show()
```

Calcul partiție optimă (repartizarea instanțelor în clusteri) prin metoda Elbow pe baza diferențelor dintre distanțele de agregare

```
distanta = z[:,2]
diferente = np.diff(distanta,2)
punct_elb = np.argmax(diferente)+1
print("Numarul optim de clustere este: ", punct_elb)

# dendograma partitie optimala
plt.figure(figsize=(10,7))
dendrogram(z, color_threshold=distanta[punct_elb-1])
plt.title("Dendrograma Partitie Optimala")
plt.show()
```

Analiza Canonica

Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
from sklearn.cross_decomposition import CCA
from scipy.stats import chi2
```

Pre-procesare

```
date = pd.read_csv('date_canonice.csv')
x = date.iloc[:, :3].values
y = date.iloc[:, 3:].values

cca = CCA(n_components=2)
cca.fit(x, y)

def test_bartlett(r2,n,p,q,m):
    x = 1-r2
    df=[(p-k+1)*(q-k+1) for k in range(1, m+1)]
    l = np.flip(np.cumprod(np.flip(x)))
    chi2_ = (-n+1+(p+q+1)/2)*np.log(1)
    return 1-chi2.cdf(chi2_, df)
```

Calcul scoruri canonice (variabile canonice)

```
x_scoruri, y_scoruri = cca.transform(x, y)
```

Calcul corelații canonice

```
corelatii_canonice = np.corrcoef(x_scoruri.T, y_scoruri.T).diagonal(offset=x_s  
print("Corelatii canonice:\n", corelatii_canonice)
```

??